Algoritmo de Dijkstra e Kruskal

ALMIR DIAS
JORGE GUILHERME
MÍRIAN NATÁLIA

Algoritmo de Kruskal

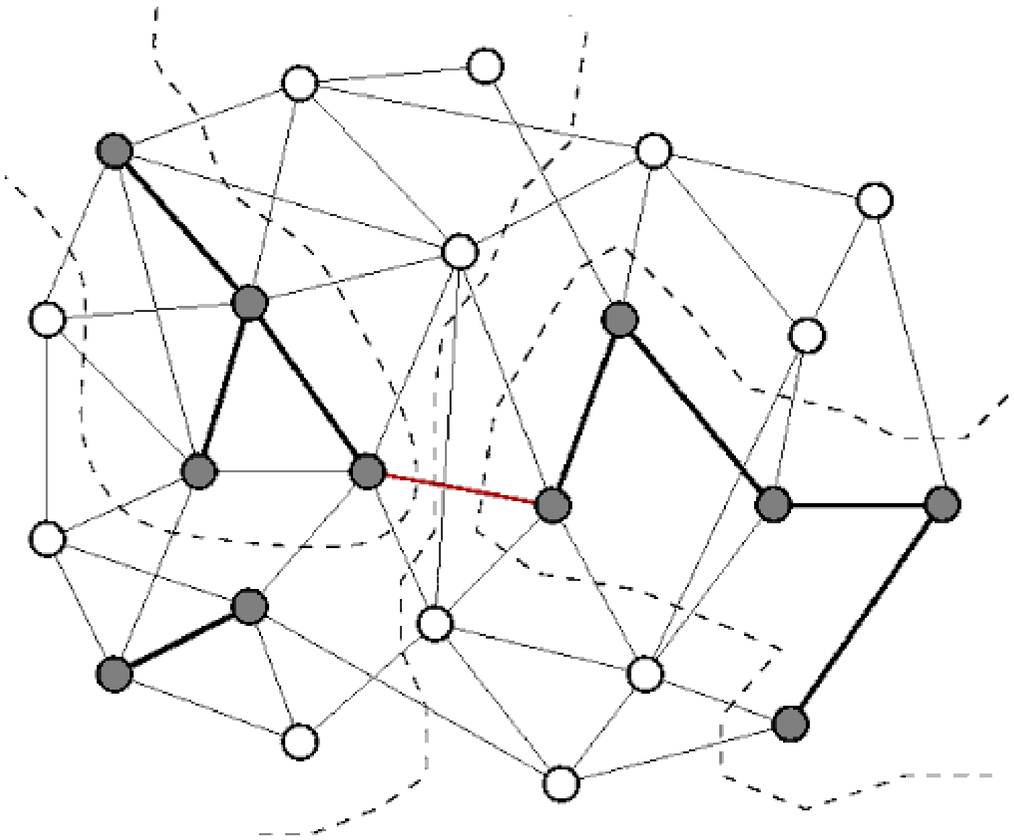
Como o algoritmo constrói a Árvore Geradora Mínima

Objetivo

Conectar todos os V vértices de um grafo G = (V, E) utilizando exatamente V - 1 arestas, sem ciclos, minimizando a soma total dos pesos.

Propriedade Fundamental

Toda aresta segura adicionada une duas componentes conexas distintas pelo menor custo possível.



Fluxo do Algoritmo de Kruskal

1 Ordenação

Ordenar todas as arestas de E em ordem crescente de peso w (qsort).

2 Seleção

Iterar pelas arestas (u, v) na ordem, até a maior disponível.

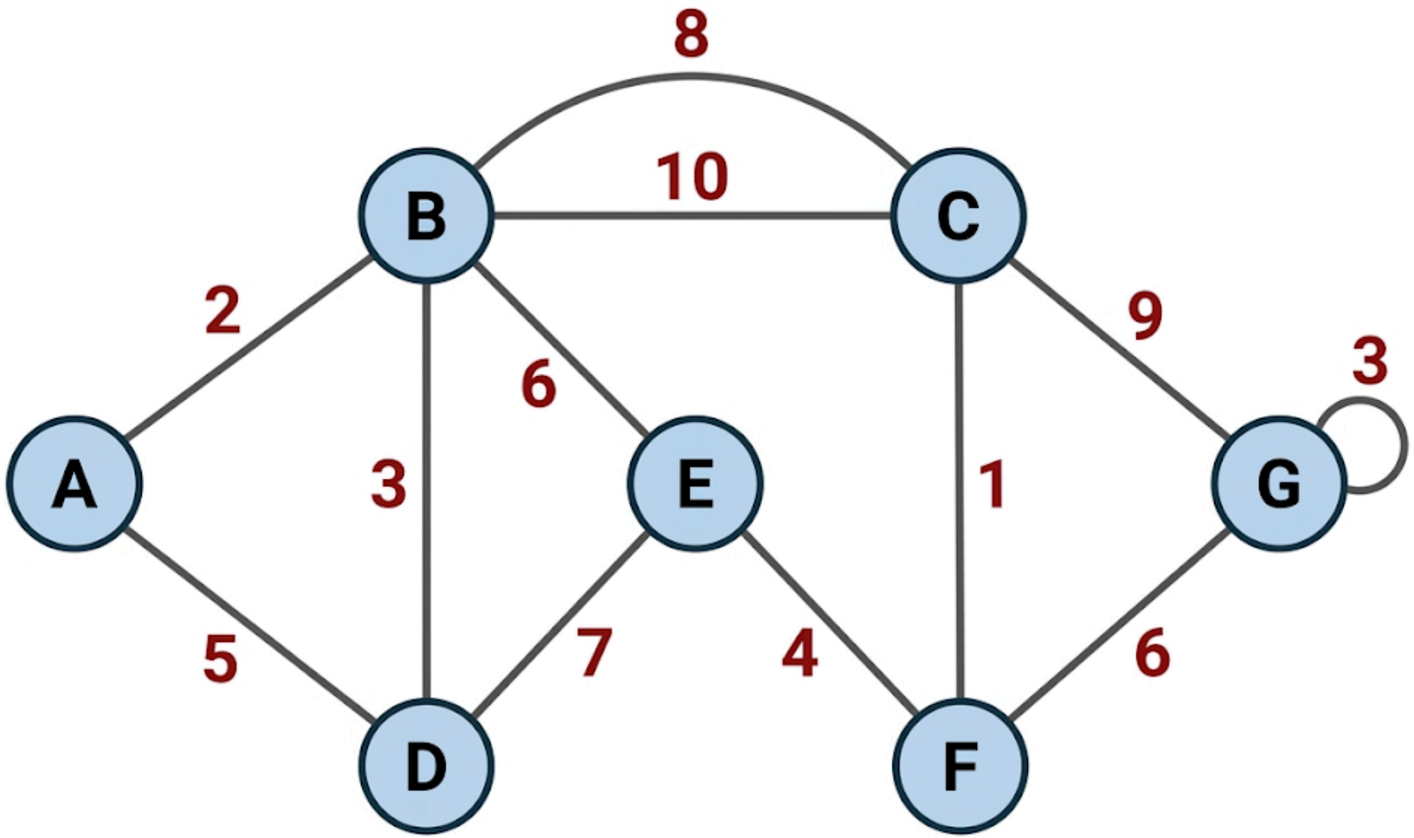
3 Teste de Ciclo

Consultar no Union-Find: Find(u) != Find(v) (estão em componentes separadas?).

4 União (MST)

Se diferentes, incluir a aresta na MST e unir subárvores com Union(u, v) até obter V - 1 arestas.

Exemplo:



Passos do algoritmo

1. Inicialização

Remova os laços e as arestas paralelas (mantenha o menor peso).

2. Extração

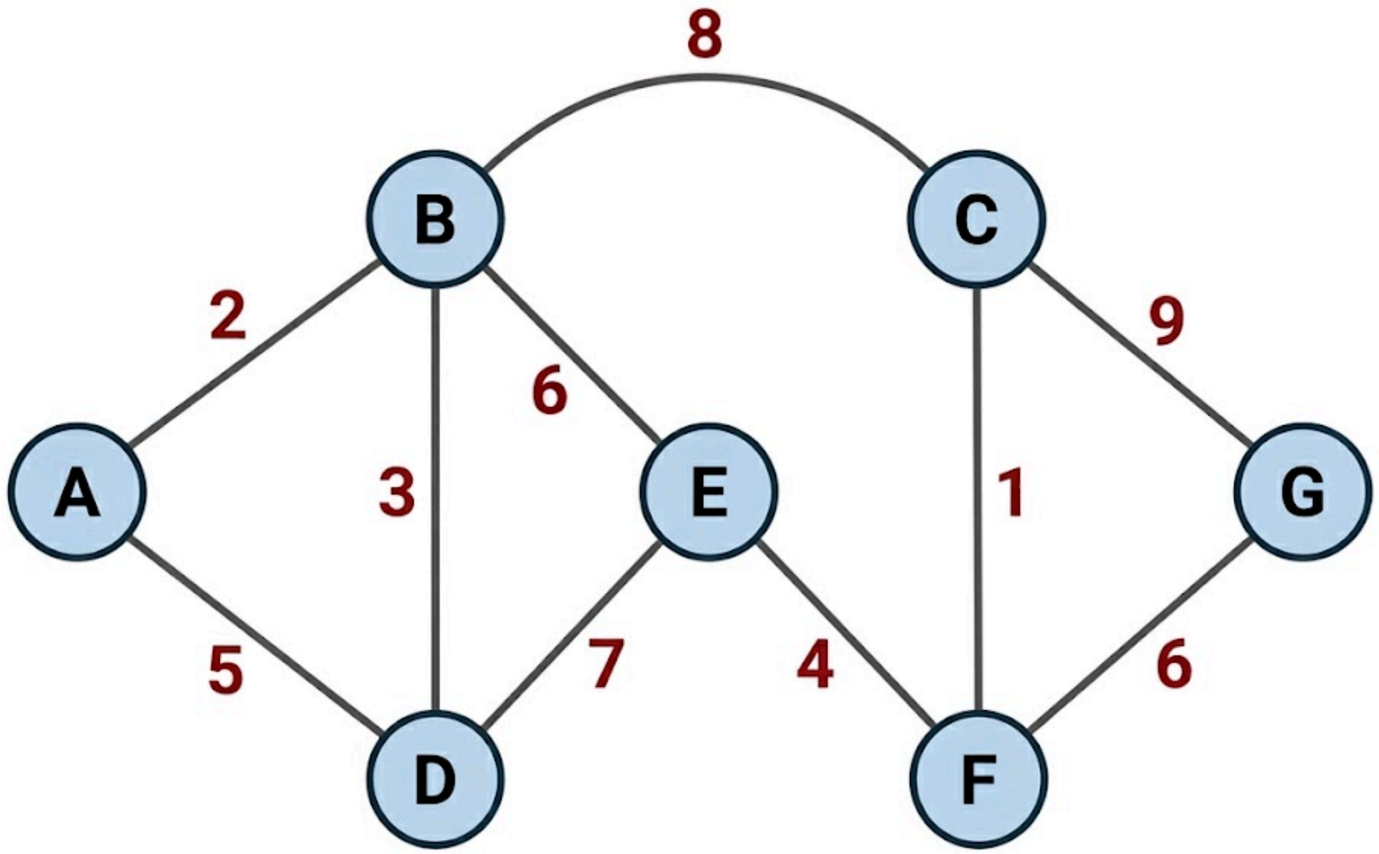
Ordene todas as arestas em ordem não decrescente de peso.

3. Relaxamento

Escolha a menor aresta e adicione-a à Árvore Geradora Mínima (AGM). (Ela não deve formar um ciclo.)

4. Atualização

Repita até que (V - 1) arestas sejam incluídas na AGM.



Pesos das arestas

Aresta	Peso
CF	1
AB	2
BD	3
EF	4
AD	5
BE	6
FG	6
DE	7
BC	8
CG	9

Implementação do Kruskal em C

```c
// 1. Ordena as arestas da mais leve para a mais pesada
merge_sort(g->arestas, 0, g->arestas_inseridas - 1);

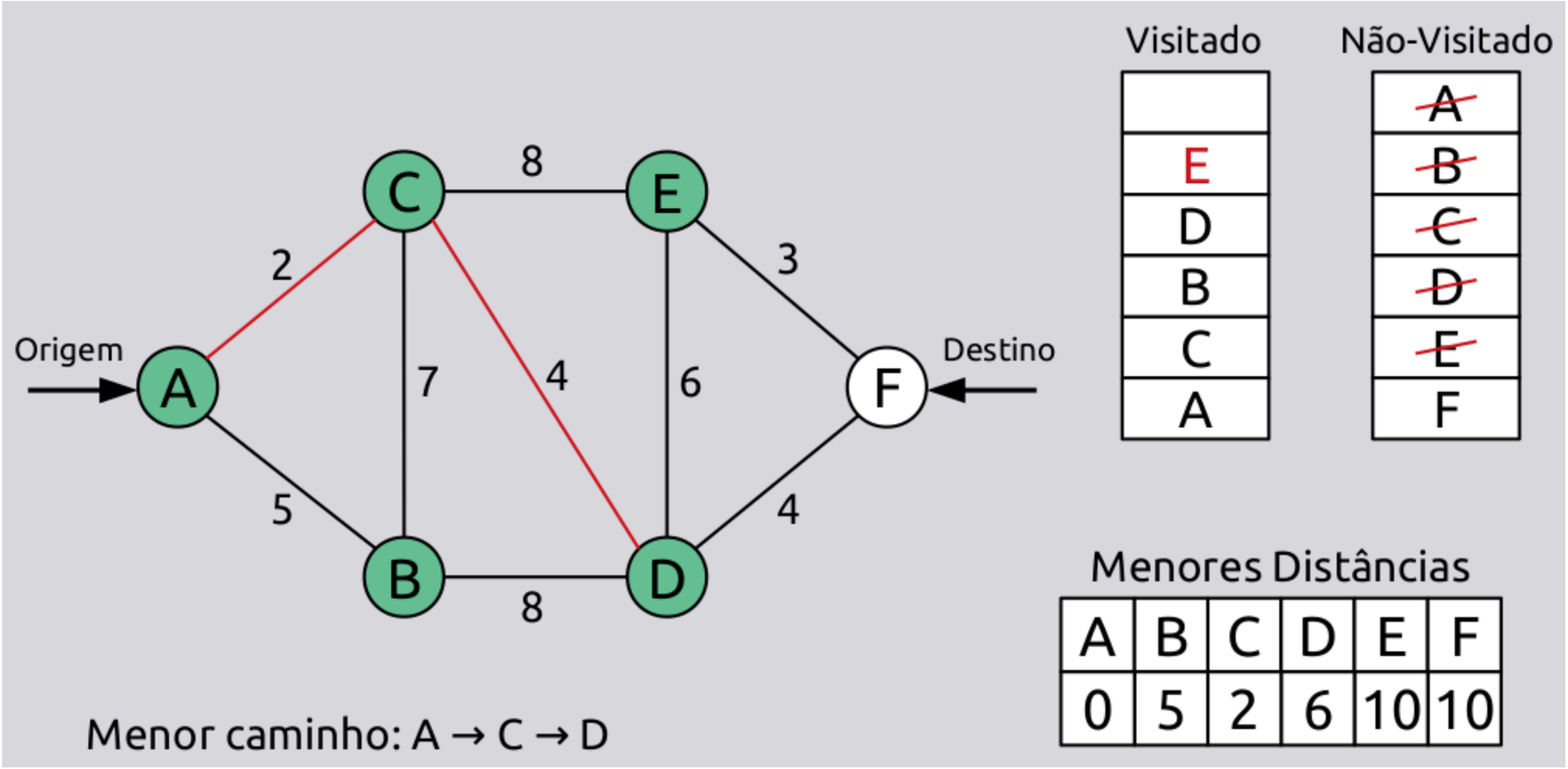
// 2. Percorre as arestas até escolher (V - 1) conexões
while (e < V - 1 && i < g->arestas_inseridas) {
    Aresta proxima = g->arestas[i++];

    // Verifica em qual grupo cada vértice está (Union-Find)
    int x = encontrar(uf, proxima.origem);
    int y = encontrar(uf, proxima.destino);

    // Se estão em grupos diferentes, a aresta não forma ciclo
    if (x != y) {
        resultado[e++] = proxima; // Adiciona na Árvore Geradora Mínima
        unir(uf, x, y);           // Junta os dois grupos num só
    }
}
```

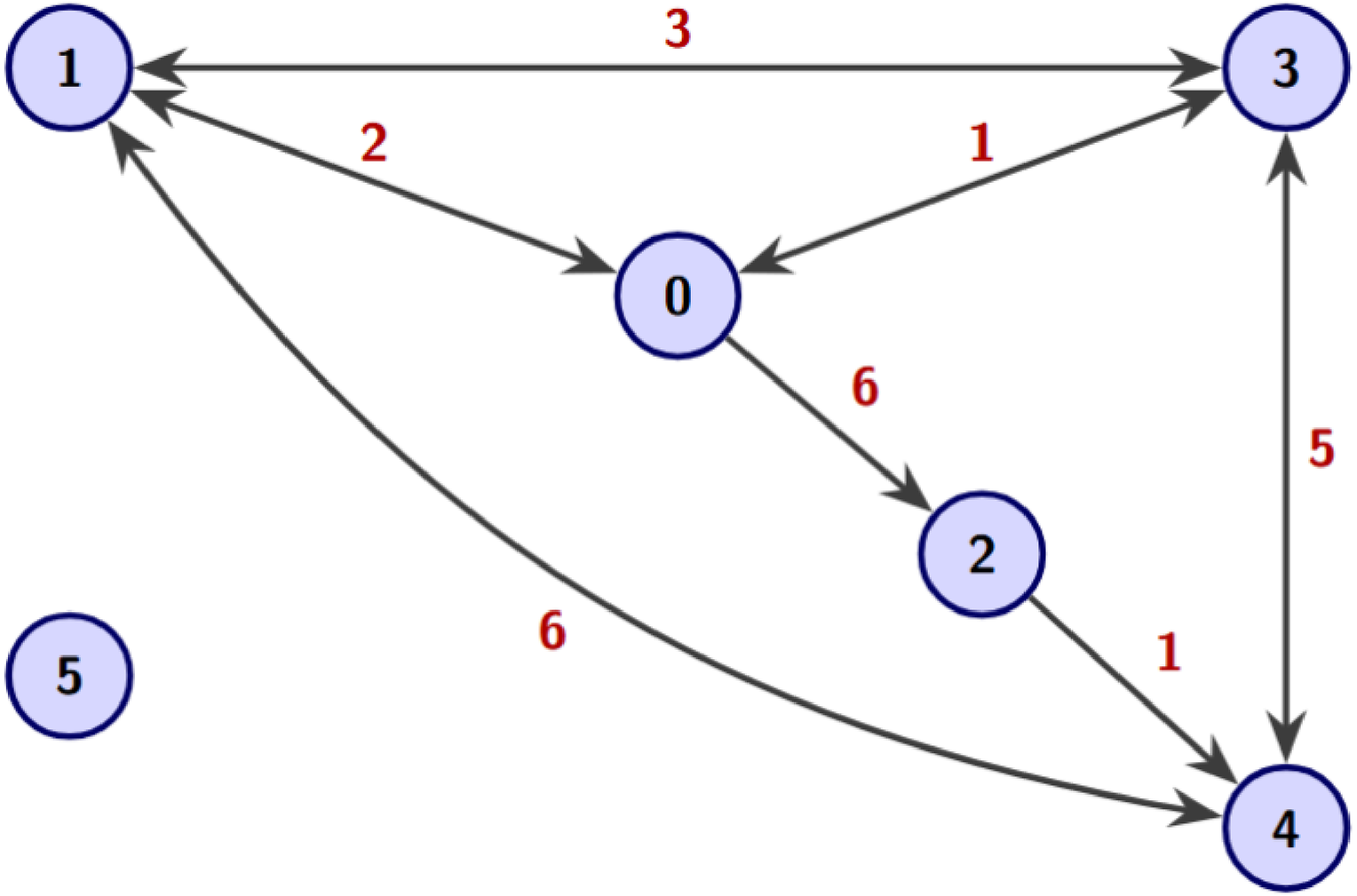
Algoritmo de Dijkstra

Determinar a menor distância do vértice de origem s até todos os demais vértices v ∈ V.



Uma vez que um vértice u é extraído da Min-Heap como o de menor distância d[u], seu caminho mínimo a partir de s já é definitivo (pois não existem pesos negativos).

Algoritmo de Dijkstra



Fluxo do Algoritmo de Dijkstra

1 Inicialização

Definir d[s] = 0, d[v] = ∞ para os demais vértices e inserir s na Min-Heap.

→

2 Extração

Remover da Min-Heap o vértice u ainda não visitado com o menor valor d[u].

→

3 Relaxamento

Para cada vizinho v de u, testar se é possível melhorar o caminho atual: d[u] + w(u, v) < d[v].

→

4 Atualização

Se menor, atualizar d[v] = d[u] + w(u, v), definir ant[v] = u e ajustar a posição de v na Min-Heap.

Implementação do Dijkstra em C

```c
// 1. Retira da Fila de Prioridade (Heap) o vértice mais próximo
NoDijkstra* atual = (NoDijkstra*)extrair_min(heap);
int u = atual->vertice;

// 2. Olha todos os vizinhos do vértice atual 'u'
NoAdj* temp = g->lista_adj[u];
while (temp != NULL) {
    int v = temp->destino;
    int peso = temp->peso;

    // 3. Relaxamento: passar por 'u' deixa o caminho até 'v' mais curto?
    if (!visitado[v] && dist[u] + peso < dist[v]) {
        dist[v] = dist[u] + peso; // Atualiza para a nova menor distância
        predecessor[v] = u;       // Anota o caminho feito
        inserir_heap(heap, novo); // Insere na fila para avaliar depois
    }
    temp = temp->proximo;
}
```